COS30049 Computing Technology Innovation Project

Assignment 2

DTDLINK
Potentially Dangerous URL Detection

Data processing classification and clustering

Session 7 Group 5

Tutor: Dr Ricky Dong

Report date: 7 October 2026

Group member	Student ID
SOKNA DAVID HEANG	105962079
DUONG DANH DAT NGUYEN	105928000
SOTHEARITH KUY	105069886

This report presents the data and machine learning part of DTDLINK. The web application is planned for Assignment 3.

1 Introduction

DTDLINK (Detect The Deception Links) helps students and other users check suspicious URLs before opening them. It analyses URL text offline and does not visit websites, perform online lookups or save submitted URLs.

Assignment 1 focused on phishing URL detection. For the Assignment 2 model, we use a wider binary target: benign or potentially dangerous. Phishing, malware and defacement are combined as the positive class. This keeps the task simple while using all four dataset labels. The model does not identify the exact attack type at prediction time.

A list of blocked URLs cannot flag a new link until it has been added. A URL text model can combine weak signals to estimate danger before a page is opened (Ma et al., 2009).

We aim to build a reusable pipeline from raw URLs to predictions and explore patterns within the dangerous class. Random Forest reaches 91.79% final test accuracy, but phishing misses remain a concern. Its output should be a warning aid, not a guarantee of safety.

2 Data source and cleaning

We used malicious_phish.csv from the course Basic_Datasets.zip. The course specification links this topic to the public Malicious URLs dataset on Kaggle (sid321axn, n.d.). We used the supplied copy rather than collecting new URLs. It contains 651,191 rows and two columns: url and type. No extra dataset was joined in this version.

Table 1 Source labels before and after cleaning

Label	Raw rows	Cleaned rows
Benign	428,103	428,074
Phishing	94,111	94,086
Malware	32,520	23,645
Defacement	96,457	95,308
Total	651,191	641,113

The audit found no missing required values, but there were repeated URLs and six URLs with conflicting benign and phishing labels. We kept the raw file unchanged and recorded a file hash, which is a fingerprint used to detect changes. The cleaning rules were applied in a fixed order.

Table 2 Cleaning rules and row changes

Rule	Result
Trim whitespace at the start or end	7 URL values changed; no rows removed
Quarantine conflicting labels	12 rows from 6 URLs set aside
Remove exact URL and label duplicates	10,066 rows removed
Create the binary target	Benign = 0; other three labels = 1

The final cleaned data has 641,113 unique URLs: 428,074 benign and 213,039 potentially dangerous. Unusual Unicode, control characters and long URLs were kept because they may carry useful information. Cleaning was run twice and produced matching output hashes. Collection dates and the original label checks are not available in the CSV.

3 Features and data splitting

We wrote an offline URL parser and feature extractor, rather than starting from an existing feature table. One URL becomes a fixed vector of 36 numeric features. The parser separates the host, path, query and other parts. It can handle missing schemes and returns issue codes for malformed input instead of crashing.

URL text features have been studied for malicious URL detection. For example, Ma et al. (2009) used properties such as URL length, host length and dots. This supports our basic lexical features, although our 36-feature design is smaller and does not reproduce their full system.

Table 3 Main feature groups

Group	Examples	Reason for using them
Lengths and counts	URL length, dots, digits, path segments	Describe the shape and size of a URL
Host and structure	IP host, subdomains, port, @ symbol	Identify address structures worth checking
Words and brand positions	login, verify, brand in subdomain	Describe potentially misleading text
Encoding and parser health	Percent encoding, Unicode, parse issues	Keep unusual input visible to the model

No single feature proves danger. HTTPS can appear in a dangerous URL, and a brand name can appear in a legitimate URL. Our Unicode look-alike check is a limited heuristic, not a full Unicode security implementation. TLD frequency was not added to the final model. Labels and domain metadata are kept for analysis but are excluded from classifier inputs.

We grouped URLs by registrable domain before splitting. This prevents URLs from the same domain appearing in both training and test data. Grouped splitting follows the idea that related examples should stay together (scikit-learn developers, n.d.a). Fourteen rows without a domain received separate URL hash groups.

Table 4 Locked domain grouped partition

Split	URLs	Purpose
Train	384,832	Fit models and preprocessing
Validation	129,332	Compare models and thresholds
Test	126,949	Evaluate the final frozen model

The split is approximately 60/20/20 with seed 42. There is zero overlap between domain groups across the splits. Model comparison used train and validation only. The test assignment was locked before modelling and used for the final evaluation after selection.

4 Data analysis

Model-directed exploration used training rows only. Benign URLs form about two thirds of the data, so accuracy alone is not enough. The train median URL length is 46 for benign and 49 for dangerous URLs. Their similar lengths show why several features are needed together.

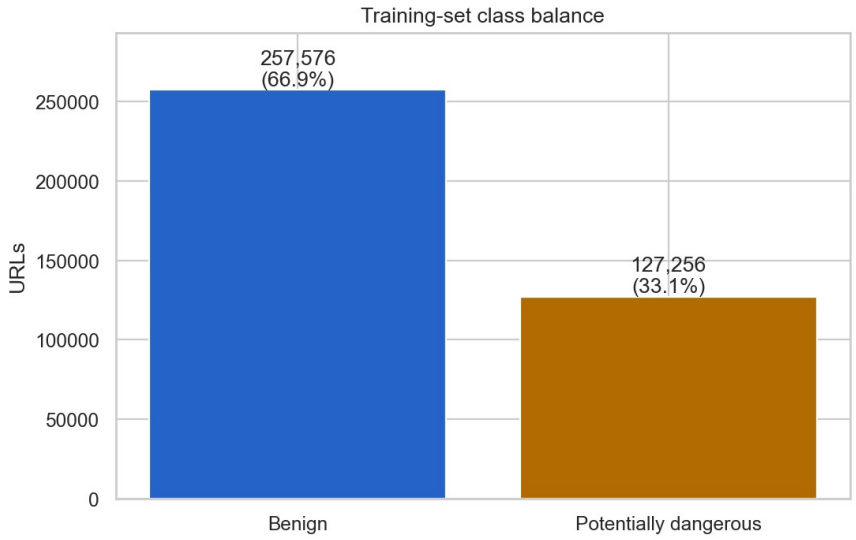


Figure 1 Class balance in training data. Source: Stage 4 saved EDA output.

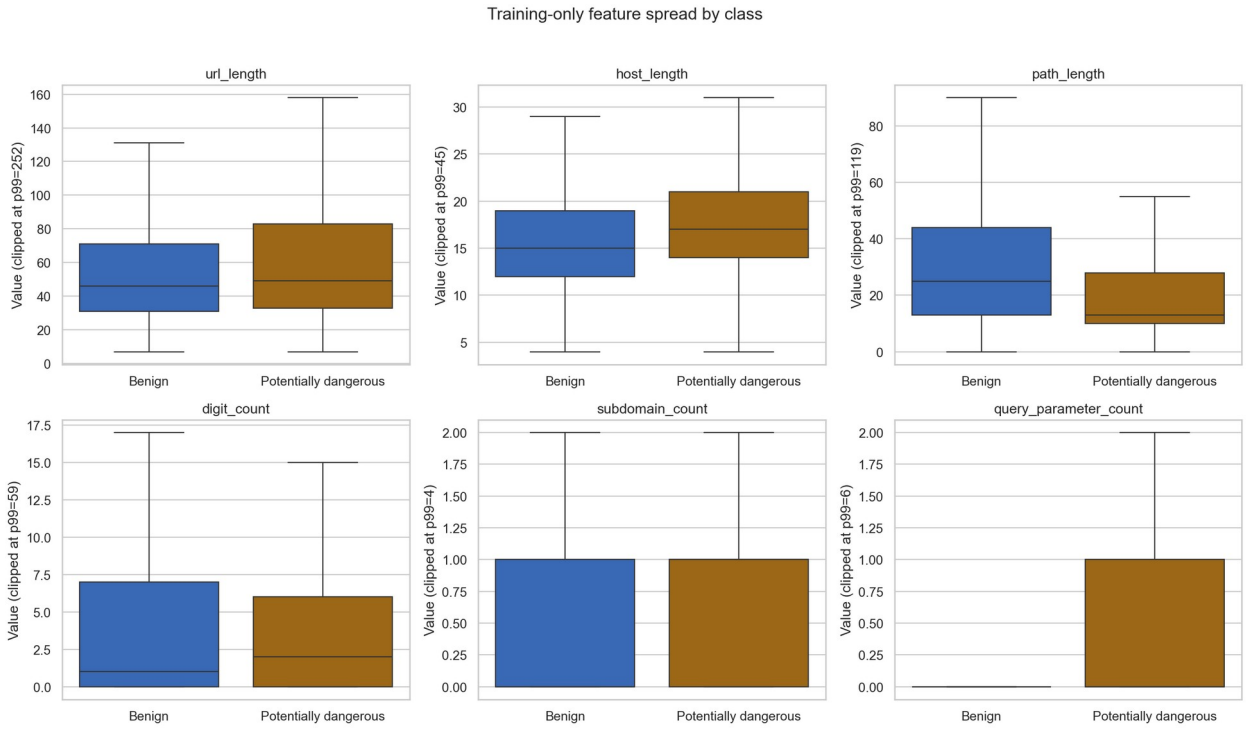


Figure 2 Feature spread by class. A fixed sample of about 50,000 training URLs is shown. Values are clipped at the training 99th percentile and outlier dots are hidden for display only. Source: Stage 4.

The two classes overlap in several features. Benign URLs also have longer paths in this sample, so a long URL alone is not enough to identify danger.

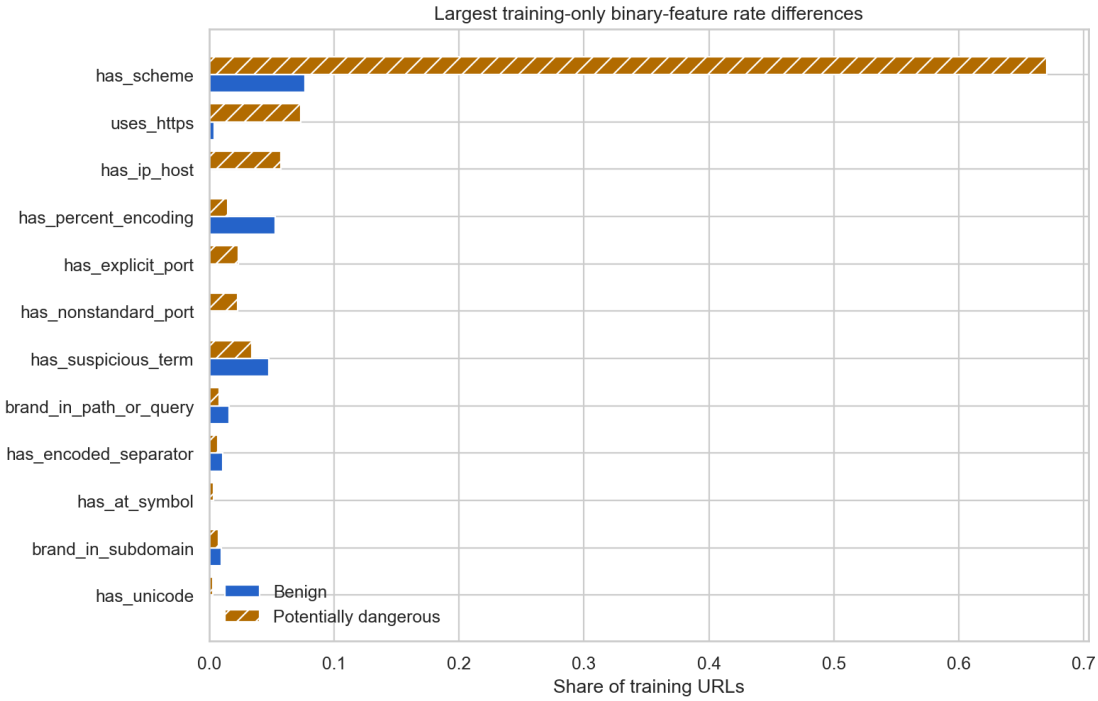


Figure 3 Rates of binary features in training data. Each bar shows the share of URLs with that feature, rather than model feature importance. Source: Stage 4.

A scheme appears in 7.66% of benign training URLs and 67.00% of dangerous training URLs. HTTPS and IP hosts are also more common in the dangerous group. These differences describe this dataset and do not prove that those features cause danger.

Formatting is a major concern. In the cleaned source audit, a scheme is written in 8.26% of benign URLs but in all defacement URLs. Missing schemes also explain many parser issues. This difference may reflect data collection rather than danger. Three feature pairs have absolute correlation above 0.85, including scheme presence and parser issue count. We kept the fixed feature schema for classification and documented these risks.

5 Classification and model choice

We compared Logistic Regression, Decision Tree and Random Forest on the same validation set. Logistic Regression is a simple linear baseline. Decision Tree can represent feature combinations. Random Forest combines many trees, which gives another way to model these relationships (Breiman, 2001).

Logistic Regression used StandardScaler fitted only on train. The tree models did not need scaling. Class weights were used to account for imbalance. Both tree models had a maximum depth of 12 and at least 100 samples per leaf. Random Forest used 100 trees, a square-root feature subset and seed 42. These limits help control model complexity.

Table 5 Validation comparison at threshold 0.5

Model	Accuracy	Precision	Recall	F1
Logistic Regression	85.34%	77.17%	79.30%	78.22%
Decision Tree	90.10%	79.82%	93.93%	86.30%
Random Forest	92.18%	84.52%	93.58%	88.82%

We selected Random Forest because it had the strongest F1 and precision while keeping high recall. Decision Tree had slightly higher recall but generated 10,195 false alarms, compared with 7,357 for Random Forest. This matters because frequent false alarms can reduce the usefulness of a warning tool.

We also reviewed thresholds from 0.2 to 0.8 on validation. At 0.4, Random Forest missed 1,837 dangerous URLs and generated 10,172 false alarms. At 0.5, it missed 2,758 and generated 7,357 false alarms. We chose 0.5 to reduce false alarms, accepting that 921 more dangerous URLs were missed than at 0.4. This is a trade-off, not a perfect setting.

The model and threshold were selected before using test. Validation results support the choice, but they are not an independent final estimate because the same validation data was used for selection.

6 Final evaluation and error analysis

The exported Random Forest was trained on train only, with the selected settings and threshold. Model comparison used validation before final test. We did not combine validation with train or use test results to choose new settings.

Table 6 Results on 126949 held out test URLs

Accuracy	Precision	Recall	F1
91.79%	85.22%	91.55%	88.27%

Precision measures how often warnings are correct; recall measures how many dangerous URLs are detected. F1 combines precision and recall.

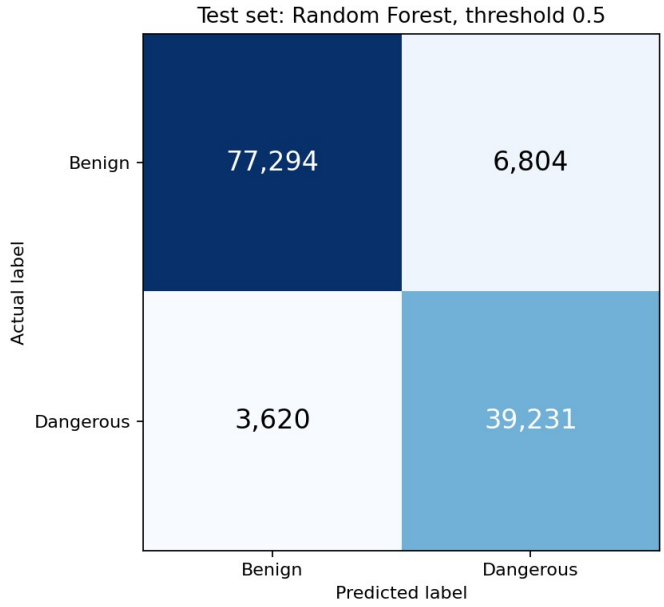


Figure 4 Final test confusion matrix. Rows are dataset labels and columns are model predictions. Source: Stage 7 saved output.

There were 77,294 correct benign predictions and 39,231 correct dangerous predictions. However, 6,804 benign URLs were incorrectly warned about and 3,620 dangerous URLs were missed. Missed dangerous URLs are especially important because a user may become too confident in a negative result.

Table 7 Recall by original dangerous label

Original label	Test URLs	Missed URLs	Recall
Phishing	18,757	3,122	83.36%
Malware	3,643	369	89.87%
Defacement	20,451	129	99.37%

Phishing is the weakest subgroup and accounts for most missed URLs. The overall dangerous recall of 91.55% must not be presented as phishing-only recall. Defacement results are much stronger, but the source formatting difference may partly affect them.

Error patterns

We saved 20 error examples, chosen by URL ID within each error type and source label. This makes the selection repeatable rather than showing only interesting cases. Missed dangerous URLs had a median length of 45, compared with 52 for detected dangerous URLs. Only 13.87% of missed dangerous URLs had a scheme, compared with 70.84% of detected dangerous URLs. These are patterns in the data, not proof that one feature caused the errors.

For example, music.yahoo.com/claudia-brant/ is labelled benign but received a dangerous probability of 0.5779. The phishing-labelled URL www.freewebs.com/swampfootrecords/ received 0.3917 and was missed. Both have short, ordinary-looking text. We did not visit these URLs or confirm their current status. Their labels are treated as dataset labels, not live security facts.

7 Clustering within the dangerous class

Clustering answers a different question from classification: what patterns exist inside one class? We selected 127,256 true-label dangerous URLs from train only. The original labels were used to choose this population, then excluded from clustering features. We also excluded URLs and domain metadata.

Two constant features were removed, leaving 34 features. StandardScaler was fitted on this population before K-means. We compared k from 2 to 8 with ten initialisations and seed 42. Silhouette compares separation between clusters and similarity within them (Rousseeuw, 1987). We used the same 2,000-row sample for every k and selected k = 7, which had the highest sampled score of 0.2923.

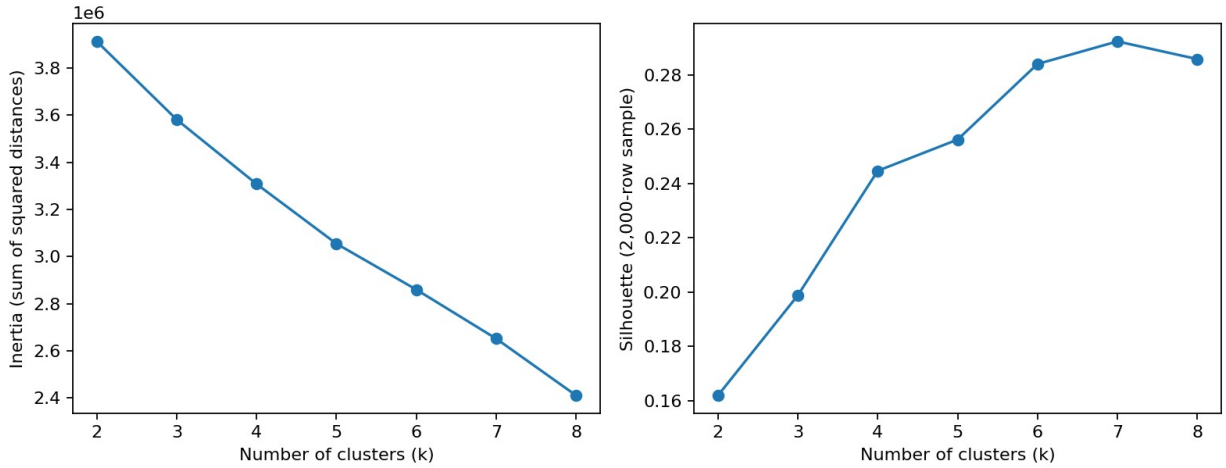


Figure 5 K-means comparison for k = 2 to 8. Inertia uses all selected URLs; silhouette uses the same 2,000-row sample. Source: Stage 6.

Inertia keeps falling as k increases. We chose seven clusters because the sampled silhouette score peaks at k = 7, rather than choosing from the inertia curve alone.

Table 8 Cluster profiles compared with the whole dangerous training class

ID	URLs	Main pattern	Cluster mean / class mean
0	40,662	Missing schemes and more parser issues	Scheme: 0.0000 / 0.6700 Parser issues: 1.0000 / 0.3305
1	26,369	Long queries with more parameters and special characters	Query parameters: 4.5379 / 1.0809 Query length: 77.8015 / 18.7317 Special characters: 19.9750 / 10.5268
2	53	Control text, unknown suffixes and non-ASCII text	Control character: 1.0000 / 0.0004 Unknown suffix: 0.6604 / 0.0003 Non-ASCII count: 47.8302 / 0.0457
3	5,730	More suspicious terms, brands and subdomains	Suspicious term: 0.7576 / 0.0344 Brand tokens: 0.3447 / 0.0179 Subdomains: 1.6332 / 0.7193
4	4,386	IP hosts without a suffix or subdomain	IP host: 1.0000 / 0.0577 Suffix length: 0.0000 / 2.8999 Subdomains: 0.0000 / 0.7193
5	47,106	Schemes present, fewer parser issues and longer paths	Scheme: 0.9941 / 0.6700 Parser issues: 0.0059 / 0.3305 Path length: 30.8032 / 20.8434
6	2,950	Nonstandard ports, IP hosts and short suffixes	Nonstandard port: 1.0000 / 0.0232 IP host: 0.9892 / 0.0577 Suffix length: 0.0325 / 2.8999

Table 8 compares URL structures with class means. Appendix A gives one representative URL per cluster, chosen near its centre. Labels did not enter K-means.

All rows are in the binary dangerous class. The original-label shares below were checked after choosing k and did not enter K-means. They are a check, rather than cluster names.

Table 9 Original label composition within each cluster. Source: Stage 6 source_type_composition.csv.

Cluster	Phishing	Malware	Defacement
0	98.49%	1.51%	0.00%
1	3.71%	2.12%	94.17%
2	100.00%	0.00%	0.00%
3	91.75%	2.83%	5.43%
4	3.42%	96.58%	0.00%
5	20.86%	14.21%	64.94%
6	0.95%	99.05%	0.00%

Clusters 0 and 5 separate missing schemes from more complete addresses. Cluster 1 contains complex queries. Cluster 2 captures rare malformed text. Cluster 3 contains more suspicious words and brand references. Clusters 4 and 6 use IP addresses, with nonstandard ports mainly in cluster 6. These patterns describe URL structure, not verified attack behaviour.

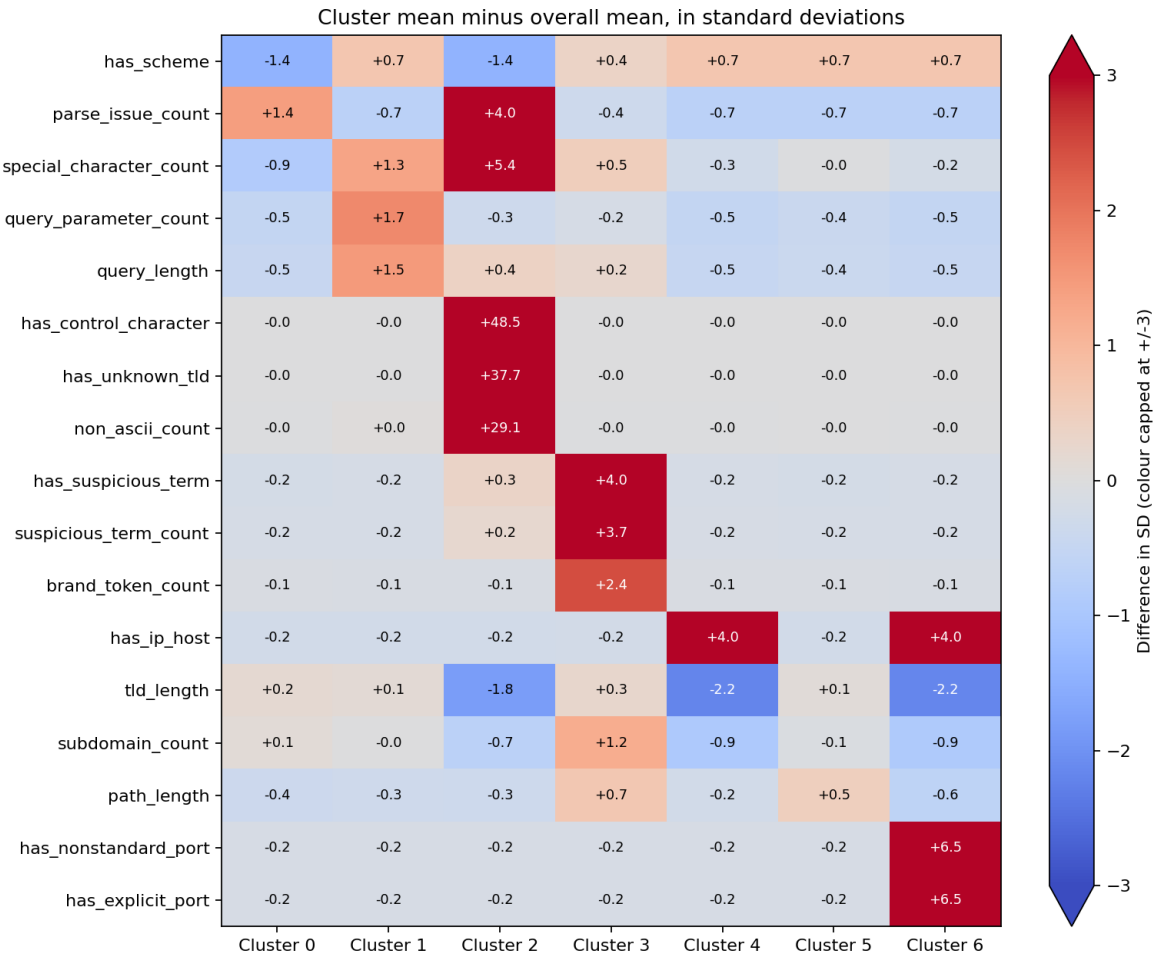


Figure 6 Cluster feature profiles relative to the whole dangerous training class. Numbers show differences in standard deviations; colours stop changing beyond -3 and +3. Source: Stage 6.

Red means a feature is higher than the class average, and blue means lower. For example, cluster 1 has more query parameters and cluster 6 has more nonstandard ports. The very large values in cluster 2 reflect rare signals. Cluster numbers are identifiers, not danger rankings.

The clustering is exploratory. The 53-row cluster had only one point in the silhouette sample. Agreement with seeds 43 and 44 was 0.9775 and 0.6016, so the grouping is not fully stable. Correlated features and scaling of rare signals can affect distances. DBSCAN was not run.

8 Implementation and verification

We used pandas and NumPy for data, scikit-learn for models, and matplotlib and seaborn for figures. Separate modules handle cleaning, parsing, features, splitting, training and prediction. tldextract supplies an offline suffix list. Handling malformed URLs and keeping domains apart were the main implementation challenges.

The export includes final_pipeline.joblib and model_metadata.json. Metadata stores the feature order, labels, threshold, model settings, split counts and model hash. Library versions are pinned in requirements.txt. The classifier has no scaler or calibration step. Source modules and compatible libraries are needed to load it.

The current development suite has 137 passing tests for unusual URLs, feature consistency and offline prediction. All 126,949 saved test probabilities matched the loaded model within 1e-12, with identical labels. We also checked 120 raw test URLs and five new inputs with network and DNS access blocked.

A fresh Python 3.9.6 environment installed the pinned requirements and reproduced processing, model comparisons, clustering and final evaluation with the fixed settings. Data hashes and result tables matched. README includes Conda setup and processing, training and prediction commands. Conda itself was unavailable here, so its configuration was checked but not run. Development tests and notebooks are omitted from the submission ZIP; the Python modules run independently.

9 Limitations and future work

The dataset has no collection dates and its labels were not independently verified. Some phishing-labelled examples look like ordinary links. Domain grouping reduces overlap but does not test future URLs or a new external source. Therefore, these results should not be treated as expected performance for every real-world URL.

The model can learn dataset formatting patterns. Its URL-only input also cannot see page content or server behaviour. Brand and suspicious-word lists are small, and the Unicode heuristic is limited. These choices keep prediction offline, but they restrict what the model can know.

The output probability is not calibrated. A value of 0.8 does not prove that 80% of similar real-world links are dangerous. Probability calibration requires a separate assessment (scikit-learn developers, n.d.b). A negative prediction should never be displayed as a guarantee that a link is safe.

Future work should use independently labelled and time-based data, investigate phishing misses, and assess calibration without tuning against the used test set. In Assignment 3, React and FastAPI should call the saved pipeline and present cautious wording and readable signals. End-to-end speed, usability and accessibility tests are still future product work.

10 Conclusion

We built a URL-text pipeline with reproducible cleaning, 36 reusable features, a domain-aware split, three classifier comparisons and clustering inside one class. Random Forest at threshold 0.5 gave the best validation F1 and achieved 88.27% F1 on final test. The analysis showed false alarms, phishing misses and source-format risks. DTDLINK now has a working ML core and a reproducible Assignment 2 package. The user-facing application remains future work.

References

Breiman, L. (2001) 'Random forests', Machine Learning, 45, pp. 5-32. doi: 10.1023/A:1010933404324. Available at: https://doi.org/10.1023/A:1010933404324 (Accessed: 6 October 2026).

Ma, J., Saul, L.K., Savage, S. and Voelker, G.M. (2009) 'Beyond blacklists: learning to detect malicious web sites from suspicious URLs', Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1245-1254. doi: 10.1145/1557019.1557153. Available at: https://cseweb.ucsd.edu/~savage/papers/KDD09.pdf (Accessed: 6 October 2026).

Rousseeuw, P.J. (1987) 'Silhouettes: a graphical aid to the interpretation and validation of cluster analysis', Journal of Computational and Applied Mathematics, 20, pp. 53-65. doi: 10.1016/0377-0427(87)90125-7.

scikit-learn developers (n.d.a) 'Cross-validation: evaluating estimator performance', scikit-learn 1.6 documentation. Available at: https://scikit-learn.org/1.6/modules/cross_validation.html (Accessed: 6 October 2026).

scikit-learn developers (n.d.b) 'Probability calibration', scikit-learn documentation. Available at: https://scikit-learn.org/stable/modules/calibration.html (Accessed: 6 October 2026).

sid321axn (n.d.) Malicious URLs dataset. Kaggle. Available at: https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset (Accessed: 6 October 2026).

Appendix A Representative cluster examples

These are dataset examples near the cluster centres. They are plain text and were not visited. The unusual string in cluster 2 is escaped and shortened for display. Full examples and feature profiles are stored in reports/clustering/.

Cluster 0

```
www.sulekhab2b.com/vastrakala/
```

Cluster 1

```
http://www.cootecsan.com/index.php?option=com_content&view=article&id=121:beneficios-adicionales&catid=55&Itemid=141
```

Cluster 2

```
\u00f6\u00c2\u0087\u00e7\u00bc\u00d2\u00e9\u00af\u00bf\u0084\u00caZ\u0089\u00ad\u0017\u00b ... [shortened]
```

Cluster 3

```
http://web196.login-4.loginserver.ch/Joomla25/templates/atomic/zyqhjnnl.php?id=1836636
```

Cluster 4

```
http://45.95.168.207/bins/jKira.m68k
```

Cluster 5

```
http://www.niobestudio.com/index.php/visita-virtual-360
```

Cluster 6

```
http://112.27.88.109:41610/Mozi.m
```

Appendix B Evidence locations

Evidence	Project location
Raw audit and cleaning	reports/audit/ and reports/cleaning/
Feature definitions	docs/feature_dictionary.md
Split and training analysis	reports/splitting/ and reports/tables/
Model comparison	reports/classification/
Cluster profiles and examples	reports/clustering/
Final metrics and errors	reports/evaluation/
Saved classifier and metadata	models/
Reusable code	src/dtdlink_ml/